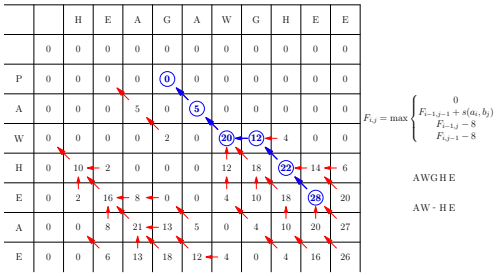
# Computational Biology

## Lecture 6: Local Sequence Matching

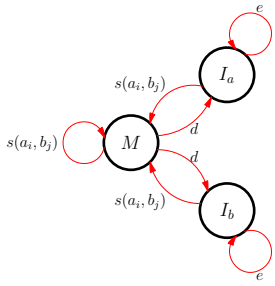



*Local sequence matching, Gaps, FSA, Extreme Values*



# Outline

1. **Local Sequence Alignment**
2. Affine Gaps
3. BLAST and FASTA
4. Significance Scores






# Local Alignment

- We saw how we could globally align sequences
- But the nature of evolution is that often sections from one part of the genome are accidentally copied to another
- Thus, we are often more interested in finding best matching sub-sequences
- In biology this problem is known as the Smith-Waterman problem after the authors that first posed the problem and gave an algorithm



# Dynamic Programming Solution

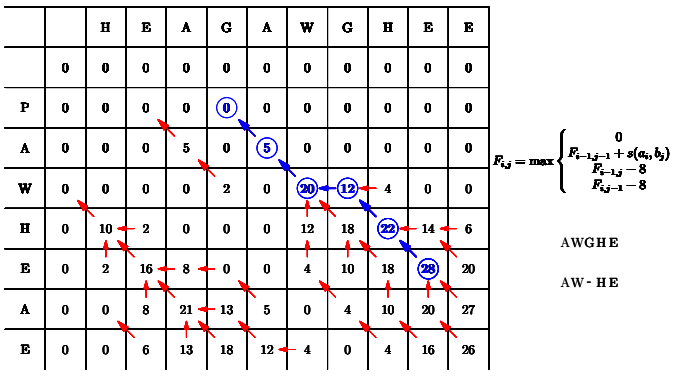
- Surprisingly, by a very small tweak to the global alignment problem, we can solve this problem too
- The trick is to add the option of ignoring the match with cost 0

$$F(i,j) = \max \begin{cases} 0 \\ F(i-1,j-1) + s(a_i, b_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \end{cases}$$

(we use $d = -8$ below)



# Local Alignment






# Difference to Global Alignment

- The forward algorithm is the same except we insist $F(i,j) \geq 0$
- We also store the location of the highest $F(i,j)$
- In the backward algorithm we start at the best value and go back until we reach 0
- It is really very easy to change a global alignment to a local alignment



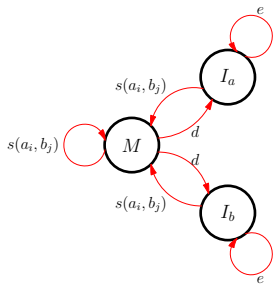
# Repeated and Overlapping Matches

- So far we have considered the single best match
- In same cases (e.g. where we have a repeated domain or motif) we may have one region being repeated many times within a second sequence
- We can modify our dynamic programming to allow this
- Again we may believe that one sequence should be completely contained in a second, then we can modify our boundary conditions to find this
- Details in *Biological sequence analysis* by Durbin *et al.*



# Outline

1. Local Sequence Alignment
2. **Affine Gaps**
3. BLAST and FASTA
4. Significance Scores



## Gaps

- As mentioned in the last lecture insertions are usually caused by adding a subsequence from elsewhere
- Deletions would result from jumping a number of bases or treating an exon as an intron
- Although these are rare events, once it happens then the gap could be quite large
- Up to now we have made a cost of a gap proportional to its length, but this seems unrealistic



## Dumb Gap Algorithm

- If we assign a cost $\gamma(g)$ to a gap of size $g$ then we could use the dynamic programme

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(a_i,b_j), & \\ \quad F(k,j) - \gamma(i-k) & k = 0,\ldots,i-1 \\ \quad F(i,k) - \gamma(j-k) & k = 0,\ldots,j-1 \end{cases}$$

- Alas for two sequences of length $n$ this is a $O(n^3)$ algorithm which is too slow for most practical applications
- Can we do better?



## Affine Gap

- If we choose a gap cost of

$$\gamma(g) = d + (g-1)\,e$$

  then there exists a $O(n^2)$ algorithm to do matching
- An affine transform of a variable $x$ is of the form $a\,x + b$ (i.e. a straight line)—note that $\gamma(g)$ is an affine function
- Using $d = -12$ and $e = -2$ gives a good model
- It penalises any gap, with larger gaps being slightly more punished



## Affine Gap Algorithm

- To efficiently cope with gaps we need to introduce three matrices (rather than just $F(i,j)$)
  - $M(i,j)$ is the best score up to $(i,j)$ given $a_i$ and $b_j$ align
  - $I_a(i,j)$ is the best score given that $a_i$ aligns with a gap
  - $I_b(i,j)$ is the best score given that $b_j$ aligns with a gap

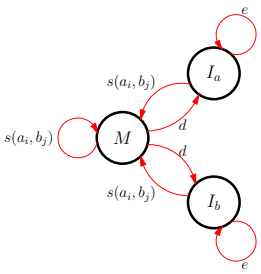


## Update Equation

$$M(i,j) = \max \begin{cases} M(i-1,j-1) + s(a_i,b_j) \\ I_a(i-1,j-1) + s(a_i,b_j) \\ I_b(i-1,j-1) + s(a_i,b_j) \end{cases}$$

$$I_a(i,j) = \max \begin{cases} M(i-1,j) - d \\ I_a(i-1,j) - e \end{cases}$$

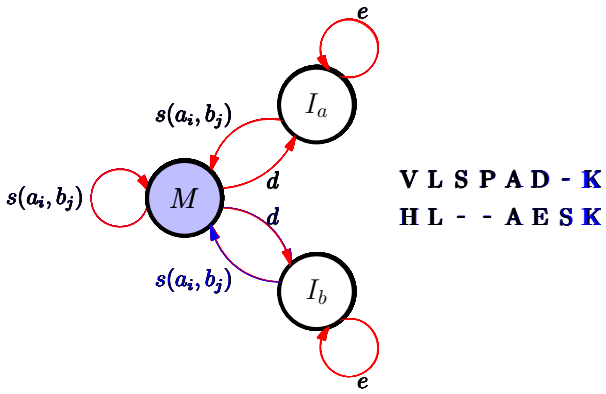
$$I_b(i,j) = \max \begin{cases} M(i,j-1) - d \\ I_b(i,j-1) - e \end{cases}$$




Finite State Automaton



## Matching with Gaps





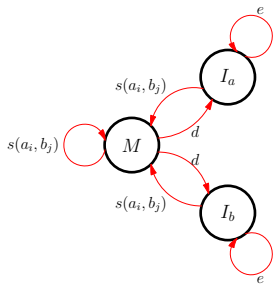

## Different FSA

- The finite state automaton (FSA) description provides an elegant visualisation of dynamic programming rules
- It makes it much easier to come up with different models that perhaps can better capture the evolutionary process
- But it comes at a cost that for each new state you need to generate an extra matrix—slowing down the matching
- In fact, for affine gaps we can come up with an approximate two state model that nearly always finds the same solution as the three state model (see Durbin et *al.*)



## Outline

1. Local Sequence Alignment
2. Affine Gaps
3. **BLAST and FASTA**
4. Significance Scores



## Inexact Matching in Practice

- Proteins are a few 100 amino acids long so $O(mn)$ is doable
- That said, there are 100's of millions of proteins in the main protein database so it can take a while to look for matches against all known proteins
- DNA is another matter. DNA sequences are of the order $10^5$ up to $10^{11}$
- Using dynamic programming on a large genome is not feasible



## Approximate Matching

- Various techniques have been developed to find approximate matches
- These are not guaranteed to find the best inexact match
- They use the heuristic of searching for sections with some exact matches
- Once these sections are identified they are extended and combined to find good matches that are polished using Smith-Waterman
- These algorithms are a rag-bag of heuristic techniques that are very effective if not very elegant



## FASTA

- FASTA is an approximate matching algorithm for both proteins and nucleotide sequences
- It firstly checks for short ($k$-mer) exact matches between a query sequence and a target ($k = 2$ for proteins and $k = 4$–$6$ for DNA)
- Local regions are identified with a high density of $k$-mers
- Local regions are extended by looking for inexact matches
- Ungapped regions are combined into gapped region
- Finally plausible regions are rematched using dynamical programming



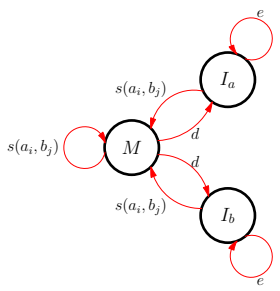
## BLAST

- BLAST (Basic Local Alignment Search Tool) is the most commonly used tool for alignment (mainly superseding FASTA)
- After removing low-complexity regions that might give high matches, exact matches (slightly larger than FASTA) are found
- BLAST builds up high scoring "words" from the exact matches
- These seeds are extended into regions which can then be combined (in BLAST2) to find gapped regions
- Again these are polished with Smith-Waterman
- For detail see Wikipedia or BLAST homepages



## Outline

1. Local Sequence Alignment
2. Affine Gaps
3. BLAST and FASTA
4. **Significance Scores**






## Are Matches Real?

- By chance we are likely to find some good matches even between random sequences
- What we would like to know is the probability of finding a particular match by chance
- This is used in algorithms like BLAST to decide which matched regions it should keep
- The problem is non-trivial because we are choosing the best match that we found

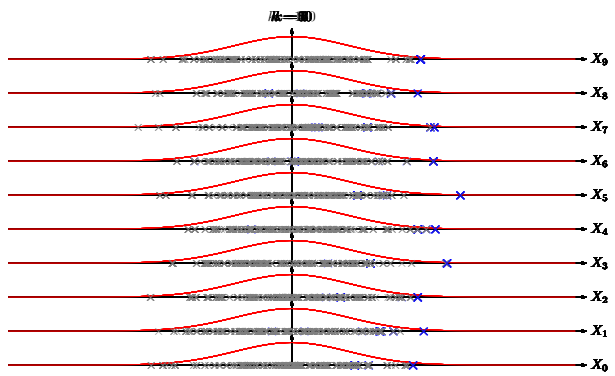


## Probability of a Match

- Computing the probability of an exact match between two random strings is possible if we know the frequency of occurrence of all the elements
- For inexact matching this requires a bit more work
- The BLOSUM50 scores are designed to give the (log-)probability of a particular match between amino acid residues
- A lot of effort has gone into creating matching schemes that give the probability of a match
- But what is the probability of the best of $k$ matches being greater than $Z$?



## Extreme Values



## Extreme Value Distribution

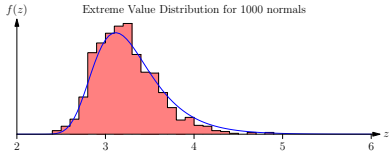
- The distribution of extreme values (best of $k$) is known as the extreme value distribution
- Similar to the central limit theorem (which states that the sum of random variables will be normally distributed), it turns out that asymptotically ($k \to \infty$) the extreme value of random deviates converges to a small number of extreme value distributions
- The most common is the Gumbel EVD given by

$$f(z) \approx \frac{1}{\beta} \mathrm{e}^{-\frac{x-\mu}{\beta} - e^{-\frac{x-\mu}{\beta}}}$$



## Using EVD

- Extreme Value Distributions become more accurate when we do a large number of trials




- In matching the number of matches are exponential in the string length so we are in a situation where EVDs are quite accurate
- The probability of observing a match with score at least $Z$ is

$$\mathbb{P}(z \geq Z) = \int_Z^\infty f(z)\,\mathrm{d}z = 1 - \exp\left(-e^{-\frac{x-\mu}{\beta}}\right)$$



## Summary

- Dynamics programming provides great flexibility in matching
- Using finite state automata we can develop more sophisticated models
- However, for speed, we often sacrifice accuracy (i.e. we might miss some good matches)
- We need ways to measure the significant of found matches